

ডেটা vs ইনফর্মেশন

Data (উপাত্ত):

Data হল অসংগঠিত ও অপরিশোধিত তথ্য। এটি অক্ষর, সংখ্যা, ছবি ইত্যাদির একটি এলোমেলো সংগ্রহ, যা নিজে থেকে কোনো নির্দিষ্ট অর্থ প্রকাশ করে না। Data-এর একককে বলা হয় Datum।

Example: Data

- 23, 45, 67, John, 78, 90, Green, Blue

এখানে বিভিন্ন সংখ্যা ও নাম রয়েছে, কিন্তু এগুলো একত্রিত হয়ে কোনো সুনির্দিষ্ট তথ্য প্রদান করছে না। উদাহরণস্বরূপ, সংখ্যা ও নামগুলোর মধ্যে কোনো সম্পর্ক বোঝা যাচ্ছে না।

Information (তথ্য):

Information হলো Data-এর একটি সংগঠিত ও অর্থবহ রূপ। এটি প্রক্রিয়াজাত Data-এর একটি সুশৃঙ্খল উপস্থাপন, যা একটি নির্দিষ্ট প্রসঙ্গে অর্থ প্রদান করে এবং সিদ্ধান্ত গ্রহণে সহায়ক।

Example: Information

- Student Name: John
- Age: 23
- Marks in Math: 78
- Marks in Science: 90
- Favorite Colors: Green, Blue

এখন, এই তথ্যগুলো একটি কর্মচারীর সম্পর্কে পরিষ্কার ধারণা দেয়। এখানে প্রতিটি উপাদান একটি নির্দিষ্ট তথ্য প্রদান করে।

এইভাবে, Data এবং Information-এর মধ্যে পার্থক্য বোঝা যায়। Data অসংগঠিত এবং অর্থহীন, যখন Information সংগঠিত এবং অর্থবহ।

Previous

মডিউল ১৪ ইন্ট্রোডাকশন

Next

ডেটাবেজ এবং ডেটাবেজের প্রকারভেদ

বিভিন্ন ধরনের Key

ডেটাবেজের বিভিন্ন ধরনের Keys নিম্নরূপ:

প্রাইমারি কি (Primary Key)

প্রাইমারি কি (Primary Key) হলো একটি ডেটাবেজের একটি বিশেষ ধরনের ফিল্ড বা কলাম, যা টেবিলের প্রতিটি রেকর্ডকে ইউনিকভাবে চিহ্নিত করে। অর্থাৎ, প্রতিটি রেকর্ডের জন্য এটি একটি অনন্য আইডেন্টিফায়ার হিসেবে কাজ করে।

Example: `EMPLOYEE` টেবিলে `EMP_ID` প্রাইমারি কি হিসেবে কাজ করে।
টেবিল: `EMPLOYEE`

EMP_ID	ENAME	DEPARTMENT	PHONE
301	Rakib	Sales	018647895479
302	Hasan	Marketing	018647895478
303	Yasin	Accounting	018647895475

কম্পোজিট প্রাইমারি কি (Composite Primary Key)

কম্পোজিট প্রাইমারি কি (Composite Primary Key) হলো একটি ডেটাবেজের প্রাইমারি কি যা দুই বা তার বেশি কলাম থেকে গঠিত হয়। যখন একটি একক কলাম একটি টেবিলের জন্য ইউনিক আইডেন্টিফায়ার হিসেবে যথেষ্ট নয়, তখন কম্পোজিট প্রাইমারি কি ব্যবহার করা হয়।

Example: `PROJECT_ASSIGNMENT` টেবিলে `{EMP_ID, PROJECT_CODE}` একটি কম্পোজিট প্রাইমারি কি হিসেবে কাজ করতে পারে।
টেবিল: `PROJECT_ASSIGNMENT`

EMP_ID	PROJECT_CODE	ROLE
301	P001	Lead
302	P002	Associate
303	P001	Associate

ফরেইন কি (Foreign Key)

ফরেইন কি (Foreign Key) হল এমন একটি অ্যাট্রিবিউট যা একটি টেবিলে অন্য টেবিলের প্রাইমারি কি'র সাথে যুক্ত থাকে। এটি দুইটি টেবিলের মধ্যে সম্পর্ক স্থাপন করে এবং ডেটার অখণ্ডতা রক্ষা করতে সহায়তা করে।

Example: `PROJECT_ASSIGNMENT` টেবিলে `EMP_ID` একটি ফরেইন কি হিসেবে কাজ করে যা `EMPLOYEE` টেবিলের `EMP_ID` কে রেফার করে।

EMP_ID	PROJECT_CODE	ROLE
301	P001	Lead
302	P002	Associate

ডেটাবেজ রিলেশনশীপ

ডেটাবেজের তিন ধরনের রিলেশনশীপ থাকেঃ

- 1. One-to-One রিলেশনশীপ
- 2. One-to-Many or Many-to-One রিলেশনশীপ
- 3. Many-to-Many রিলেশনশীপ

One-to-One রিলেশনশীপ

One-to-One রিলেশনশীপ হলো ডেটাবেজের দুটি টেবিলের মধ্যে এমন একটি সম্পর্ক যেখানে একটি টেবিলের একটি রেকর্ড শুধুমাত্র অপর টেবিলের একটি রেকর্ডের সাথে সম্পর্কিত হয়। অর্থাৎ, প্রতিটি রেকর্ডের জন্য অন্য টেবিলে শুধুমাত্র একটি সম্পর্কিত রেকর্ড থাকবে।

Example: প্রতি Seller একটি নির্দিষ্ট Product থাকতে পারে এবং প্রতিটি Product শুধুমাত্র এক Seller এর সঙ্গে সম্পর্কিত। তাই এটি One-to-One রিলেশনশীপ।

Seller Table

SellerID	SellerName	Address
1	Jahin	Dhaka
2	Rita	Chittagong

Product Information Table

CustomerID	CustomerName
1	Rofiq
2	Kamal

One-to-Many or Many-to-One রিলেশনশীপ

One-to-Many or Many-to-One রিলেশনশীপ হলো ডেটাবেজের দুটি টেবিলের মধ্যে সম্পর্ক, যেখানে একটি টেবিলের একটি রেকর্ড অপর টেবিলের একাধিক রেকর্ডের সাথে সম্পর্কিত হতে পারে।

Example: প্রতিটি Customer এর একাধিক Order থাকতে পারে। এটি একটি One-to-Many রিলেশনশীপ। তবে বিপরীতভাবে, একাধিক Order এরও এক Customer এর অধীনে হতে পারে, তাই এটি একটি Many-to-One রিলেশনশীপ।

Customer Table

CustomerID	CustomerName
1	Rofiq
2	Kamal

Order Table

OrderID	CustomerID	ProductName
1	1	Notebook
2	1	Pen
3	2	Book

Many-to-Many রিলেশনশীপ

Many-to-Many রিলেশনশীপ হলো একটি ডেটাবেজের দুটি টেবিলের মধ্যে এমন একটি সম্পর্ক যেখানে একটি টেবিলের একাধিক রেকর্ড অপর টেবিলের একাধিক রেকর্ডের সাথে সম্পর্কিত হতে পারে। এই ধরনের সম্পর্কগুলি সাধারণত একটি মধ্যবর্তী টেবিলের মাধ্যমে প্রতিষ্ঠিত হয়।

Example: প্রতিটি Student একাধিক Courses এ Enroll করতে পারে, এবং একাধিক Course তেও একাধিক Student থাকতে পারে। এটি একটি Many-to-Many রিলেশনশীপ।

Students Table

StudentID	StudentName
1	Rahul
2	Sohel
3	Tanima

Courses Table

SellerID	SellerName	Address
1	Jahin	Dhaka
2	Rita	Chittagong

Enrollments Table

EnrollmentID	StudentID	CourseID
1	1	1
2	1	3
3	2	2
4	3	1
5	3	2
6	3	3

কোয়েরি

কোয়েরি কী?

কোয়েরি হল একটি নির্দিষ্ট নির্দেশনা, যা ডেটাবেজ থেকে তথ্য আনতে বা ম্যানিপুলেট করতে ব্যবহৃত হয়।\

বিভিন্ন ধরনের কোয়েরি নিয়ে আলোচনা করা হলঃ

Select Query

এটি মূলত একটি তথ্য বের করার কোয়েরি। ধরো, তুমি কোনো নির্দিষ্ট শিক্ষার্থীর রোল নম্বর জানো (যেমন, 102)। তখন সিলেক্ট কোয়েরি ব্যবহার করে সেই রোল অনুযায়ী শিক্ষার্থীর নাম বা অন্য তথ্য খুঁজে বের করতে পারবে।

Example:

- একটি টেবিলের নাম: Student
- রোল: 102
- ফলাফল: Rahim

ব্যবহার: বড় ডেটাবেজ থেকে নির্দিষ্ট তথ্য আনতে।

Parameter Query

প্যারামিটার কোয়েরি এমন একটি কোয়েরি যা ইউজারের কাছ থেকে ইনপুট নেয়। ধরো, তুমি 102 নম্বর রোল দিয়ে তথ্য খুঁজবে। কিন্তু তুমি চাইছো, পরবর্তীতে ভিন্ন রোল নম্বর দিয়েও অন্য শিক্ষার্থীদের তথ্য বের করতে। তখন এই প্যারামিটার কোয়েরি প্রতিবার নতুন প্যারামিটার হিসেবে ইনপুট চায়।

Example:

- ইনপুট প্যারামিটার: রোল 102

ব্যবহার: ডায়নামিক কোয়েরি করার জন্য এইটি ব্যবহার করা হয়, যেখানে ইউজারের ইনপুটের উপর ভিত্তি করে তথ্য বের করা হয়ে থাকে।

Crosstab Query

ক্রসট্যাব কোয়েরি এমন একটি কোয়েরি, যা টেবিলের তথ্যগুলোকে সারসংক্ষেপ আকারে দেখায় বা গ্রুপ করে দেখায়।

Example: তোমার কাছে বিভিন্ন শিক্ষার্থীর প্রাপ্ত নম্বরের ডেটা আছে, আর তুমি চাও প্রতিটি বিষয়ের গড়ে কত নম্বর পাওয়া হয়েছে সেটি দেখতে। তখন ক্রসট্যাব কোয়েরি তোমাকে এক ধরনের পিভট টেবিলের মত আউটপুট দেবে। এভাবে বিভিন্ন বিষয় অনুযায়ী গ্রুপিং করে ফলাফল দেখানোকে পিভট টেবিল বলা হয়।

ব্যবহার: বড় পরিমাণ তথ্যকে বিশ্লেষণ করার জন্য।

Unmatched Query

আনম্যাচড কোয়েরি এমন একটি কোয়েরি, যা দুইটি টেবিলের মধ্যে মিল খুঁজে না পাওয়া রেকর্ডগুলো বের করে।

Example: ধরো, তোমার একটি টেবিলে শিক্ষার্থীদের রোল নম্বর আছে আর অন্য একটি টেবিলে তাদের পরীক্ষার রেজাল্ট আছে। তুমি আনম্যাচড কোয়েরি দিয়ে বের করতে পারো, কোন রোল নম্বরগুলোতে এখনো কোনো রেজাল্ট , রেজাল্ট টেবিলে যোগ করা হয়নি।

ব্যবহার: যেকোনো ধরণের ডেটাবেজে মিসিং ডেটা বা অসম্পূর্ণ তথ্য খুঁজে বের করার জন্য কাজে লাগে।

Action Query

অ্যাকশন কোয়েরি শুধু তথ্য বের করেই থেমে থাকে না, বরং ডেটাবেজে পরিবর্তন বা মডিফিকেশন আনে। ধরো, তোমার কাছে একটি টেবিলে ছাত্রদের নাম আছে এবং তুমি সবার নামের শেষে "Mr." যোগ করতে চাও। তখন Update Query দিয়ে এই কাজ করা যায়।

অ্যাকশন কোয়েরি কয়েক ধরনের কাজ করতে পারে, যেমন:

- Make Table:** ডেটাবেজ থেকে তথ্য নিয়ে নতুন একটি টেবিল তৈরি করে।
- Append:** একটি টেবিলে নতুন তথ্য যুক্ত করে।
- Update:** আগের কোনো তথ্য পরিবর্তন করে।

ব্যবহার: ডেটাবেজের তথ্য পরিবর্তন বা ম্যানিপুলেট করার জন্য।

MySQL Datatypes

String Data Types

Data Type	Description	Range/Max Length	Example
CHAR(size)	Fixed-length string	0 to 255 characters	CHAR(10) (stores 10 characters, padding with spaces)
VARCHAR(size)	Variable-length string	0 to 65,535 characters	VARCHAR(255) (up to 255 chars, using necessary space)
BINARY(size)	Fixed-length binary byte string	0 to 255 bytes	BINARY(5) (stores exactly 5 bytes)
VARBINARY(size)	Variable-length binary byte string	0 to 65,535 bytes	VARBINARY(255) (up to 255 bytes)
TINYBLOB	Binary Large Object	Max 255 bytes	Useful for small binary files
TINYTEXT	String	Max 255 characters	Ideal for short textual content
TEXT	String	Max 65,535 bytes	Suitable for longer text
MEDIUMTEXT	String	Max 16,777,215 characters	Good for lengthy documents
LONGTEXT	String	Max 4,294,967,295 characters	Ideal for massive text data
ENUM(val1, val2, ...)	A string object with one value from a list	Up to 65,535 values	ENUM('small', 'medium', 'large')
SET(val1, val2, ...)	A string object with 0 or more values from a list	Up to 64 values	SET('red', 'green', 'blue')

Numeric Data Types

Data Type	Description	Range	Example
BIT(size)	Bit-value type	1 to 64 bits	BIT(8) (stores an 8-bit value)
TINYINT(size)	Very small integer	Signed: -128 to 127; Unsigned: 0 to 255	TINYINT(3)
BOOL/BOOLEAN	Synonym for TINYINT(1)	-	BOOLEAN (true: 1, false: 0)
SMALLINT(size)	Small integer	Signed: -32,768 to 32,767; Unsigned: 0 to 65,535	SMALLINT(5)
MEDIUMINT(size)	Medium integer	Signed: -8,388,608 to 8,388,607; Unsigned: 0 to 16,777,215	MEDIUMINT(8)
INT/INTEGER(size)	Standard integer	Signed: -2,147,483,648 to 2,147,483,647; Unsigned: 0 to 4,294,967,295	INT(11)
BIGINT(size)	Large integer	Signed: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807; Unsigned: 0 to 18,446,744,073,709,551,615	BIGINT(20)
FLOAT(size, d)	Floating-point number	-	FLOAT(7, 4) (deprecated in 8.0.17)
DOUBLE(size, d)	Normal-size floating-point number	-	DOUBLE(16, 8)
DECIMAL(size, d)	Exact fixed-point number	Max size: 65; Max d: 30	DECIMAL(10, 2) (monetary values)

Date and Time Data Types

Data Type	Description	Format/Range	Example
DATE	Date value	YYYY-MM-DD; '1000-01-01' to '9999-12-31'	DATE
DATETIME(fsp)	Date and time	YYYY-MM-DD hh:mm; '1000-01-01 00:00:00' to '9999-12-31 23:59:59'	DATETIME(3) (milliseconds)
TIMESTAMP(fsp)	Timestamp	YYYY-MM-DD hh:mm; '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
TIME(fsp)	Time value	hh:mm; '-838:59:59' to '838:59:59'	TIME(3) (fractional seconds)
YEAR	Four-digit year	1901 to 2155, and 0000	YEAR

CREATE, INSERT, UPDATE, DELETE, DROP

CREATE

CREATE কমান্ড একটি নতুন টেবিল ডেটাবেজে define করতে ব্যবহৃত হয়। টেবিলের নাম, এর কলামগুলো এবং প্রতিটি কলামের ডেটা টাইপ নির্ধারণ করা হয়।

Syntax:

```
1 CREATE TABLE table_name (  
2   column1 datatype,  
3   column2 datatype,  
4   ...  
5 )
```

Example:

```
1 CREATE TABLE employees (  
2   id INT AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(50),  
4   position VARCHAR(50)  
5 );
```

Output: টেবিল employees সফলভাবে তৈরি হয়েছে, যার তিনটি কলাম: id, name, এবং position।

INSERT

INSERT কমান্ড একটি টেবিলের মধ্যে নতুন তথ্য add করে।

Syntax:

```
1 INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
```

Example:

```
1 INSERT INTO employees (name, position) VALUES ('Alice', 'Developer');  
2 INSERT INTO employees (name, position) VALUES ('Bob', 'Manager');
```

Output:

id	name	position
1	Alice	Developer
2	Bob	Manager

UPDATE

UPDATE কমান্ড দিয়ে একটি টেবিলের বিদ্যমান রেকর্ডগুলোকে পরিবর্তন করা যায়।

Syntax:

```
1 UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;
```

Example:

```
1 UPDATE employees SET position = 'Senior Developer' WHERE name = 'Alice';
```

Output:

id	name	position
1	Alice	Developer
2	Bob	Manager

DELETE

DELETE কমান্ড একটি টেবিল থেকে রেকর্ডগুলো সরিয়ে দেয় নির্দিষ্ট শর্তের ভিত্তিতে।

Syntax:

```
1 DELETE FROM table_name WHERE condition;
```

Example:

```
1 DELETE FROM employees WHERE name = 'Bob';
```

id	name	position
1	Alice	Senior Developer

DROP

DROP কমান্ড একটি টেবিল এর সমস্ত তথ্য ডেটাবেজ থেকে স্থায়ীভাবে মুছে ফেলে।

Syntax:

```
1 DROP TABLE table_name;
```

Example:

```
1 DROP TABLE employees;
```

Output: employees টেবিল ডেটাবেজ থেকে সরিয়ে ফেলা হয়েছে। যদি আপনি পরে এটি অ্যাক্সেস করার চেষ্টা করেন, তাহলে আপনি একটি এরর পাবেন:

```
1 SELECT * FROM employees;
```

Error: Table 'database_name.employees' doesn't exist